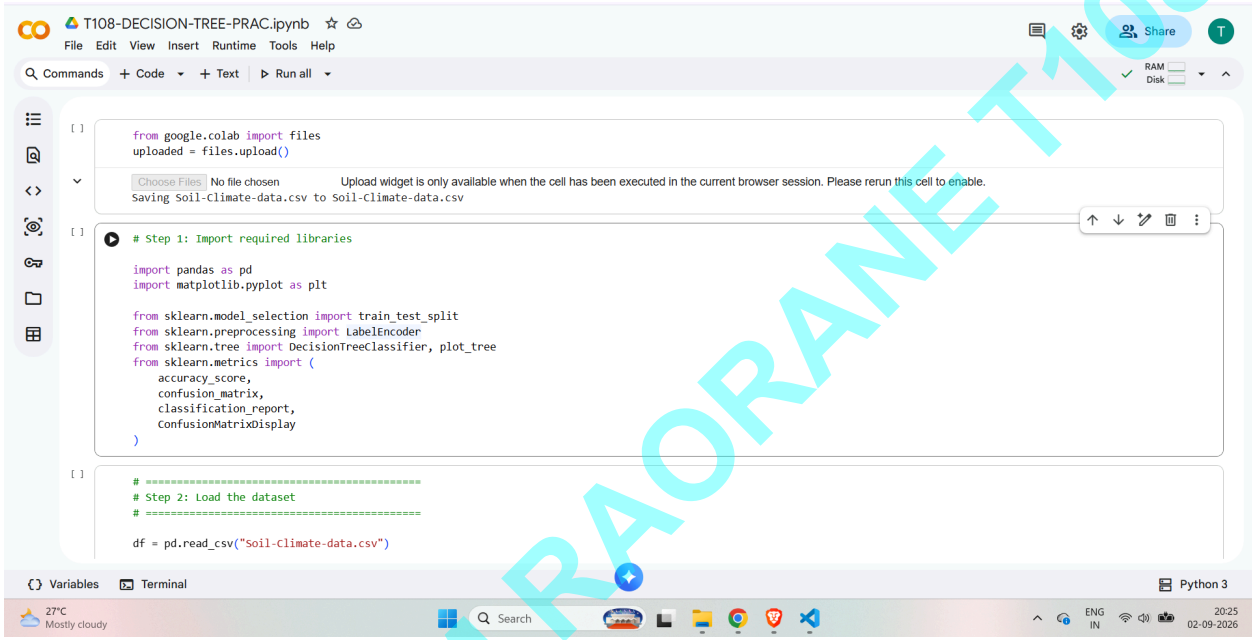
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

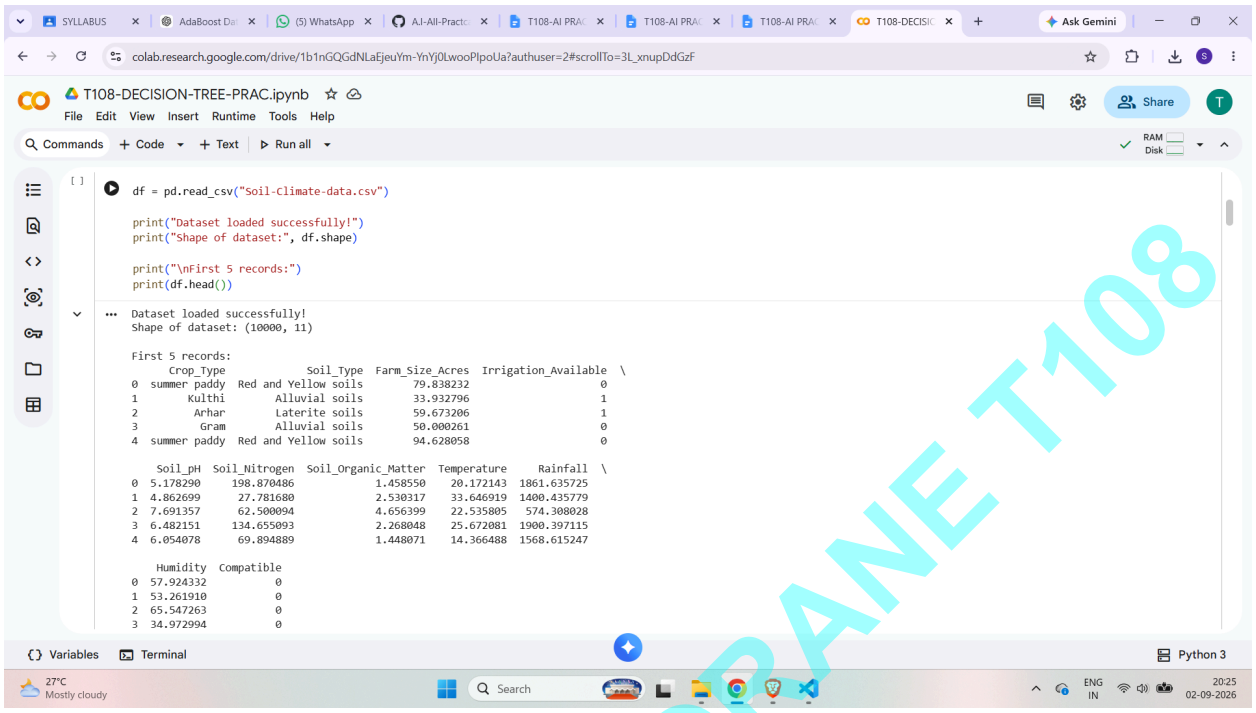
AIM : Decision Tree Learning
Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
Evaluate the accuracy and effectiveness of the decision tree on test data.
Visualize and interpret the generated decision tree.

Import required libraries



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

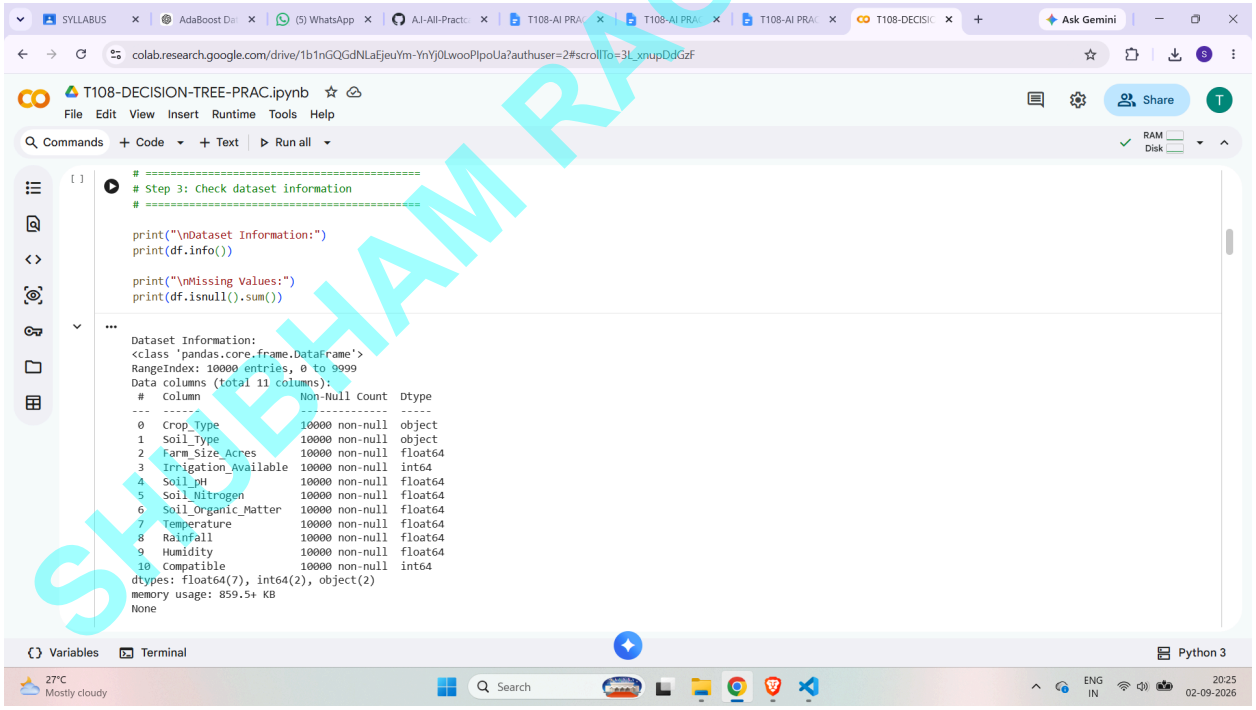
Load the dataset



```
df = pd.read_csv("Soil-Climate-data.csv")

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)

print("\nFirst 5 records:")
print(df.head())
```

```
Dataset loaded successfully!
Shape of dataset: (10000, 11)

First 5 records:
      Crop_Type            Soil_Type  Farm_Size_Acres  Irrigation_Available  \
0  summer paddy  Red and Yellow soils        79.838232                     0
1        Kulthi        Alluvial soils        33.932796                     1
2         Arhar        Laterite soils        59.673206                     1
3          Gram        Alluvial soils        50.000261                     0
4  summer paddy  Red and Yellow soils        94.628058                     0

    Soil_pH  Soil_Nitrogen  Soil_Organic_Matter  Temperature     Rainfall  \
0  5.178290     198.870486             1.458550    20.172143  1861.635725
1  4.862699      27.781680             2.530317    33.646919  1400.435779
2  7.691357      62.500094             4.656399    22.535805   574.308028
3  6.482151     134.655093             2.268048    25.672081  1900.397115
4  6.054078      69.894889             1.448071    14.366488  1568.615247

    Humidity  Compatible
0  57.924332           0
1  53.261910           0
2  65.547263           0
3  34.972994           0
```

Check dataset information

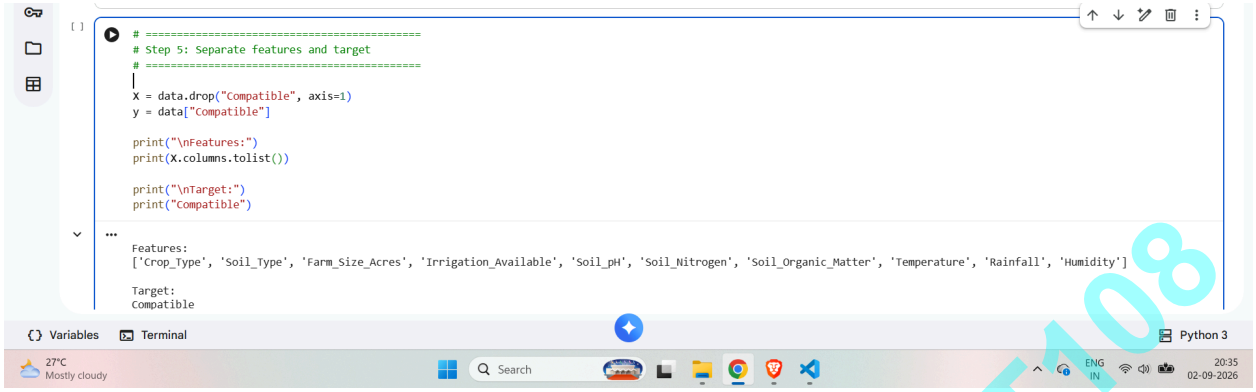


```
# ============================================
# Step 3: Check dataset information
# ============================================

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Crop_Type             10000 non-null  object
 1   Soil_Type             10000 non-null  object
 2   Farm_Size_Acres       10000 non-null  float64
 3   Irrigation_Available  10000 non-null  int64
 4   Soil_pH               10000 non-null  float64
 5   Soil_Nitrogen         10000 non-null  float64
 6   Soil_Organic_Matter   10000 non-null  float64
 7   Temperature           10000 non-null  float64
 8   Rainfall              10000 non-null  float64
 9   Humidity              10000 non-null  float64
 10  Compatible            10000 non-null  int64
dtypes: float64(7), int64(2), object(2)
memory usage: 859.5+ KB
None
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

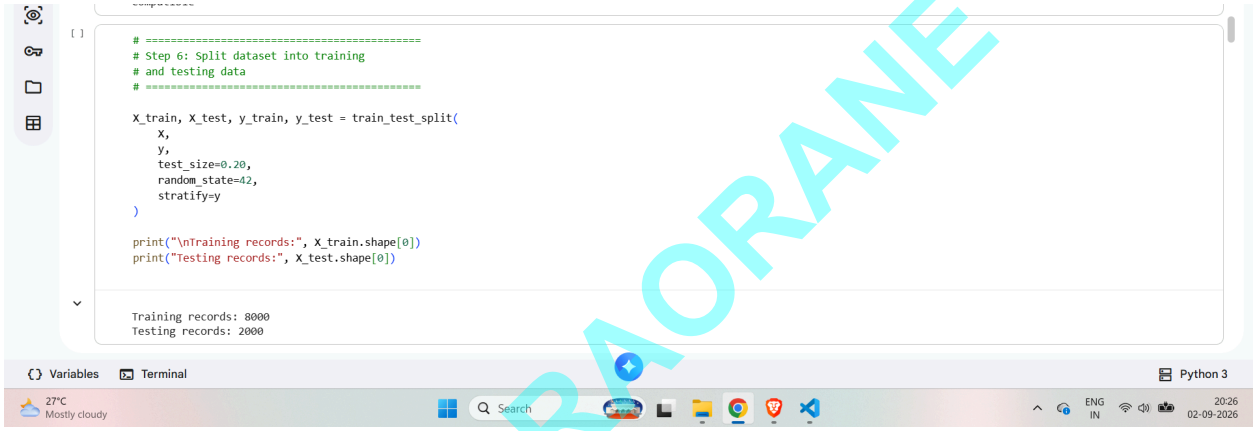
Separate features and target



```
# ============================================
# Step 5: Separate features and target
# ============================================

X = data.drop("Compatible", axis=1)
y = data["Compatible"]

print("\nFeatures:")
print(X.columns.tolist())

print("\nTarget:")
print("Compatible")
```

```
Features:
['Crop_Type', 'Soil_Type', 'Farm_Size_Acres', 'Irrigation_Available', 'Soil_pH', 'Soil_Nitrogen', 'Soil_Organic_Matter', 'Temperature', 'Rainfall', 'Humidity']

Target:
Compatible
```

Split dataset into training and testing data

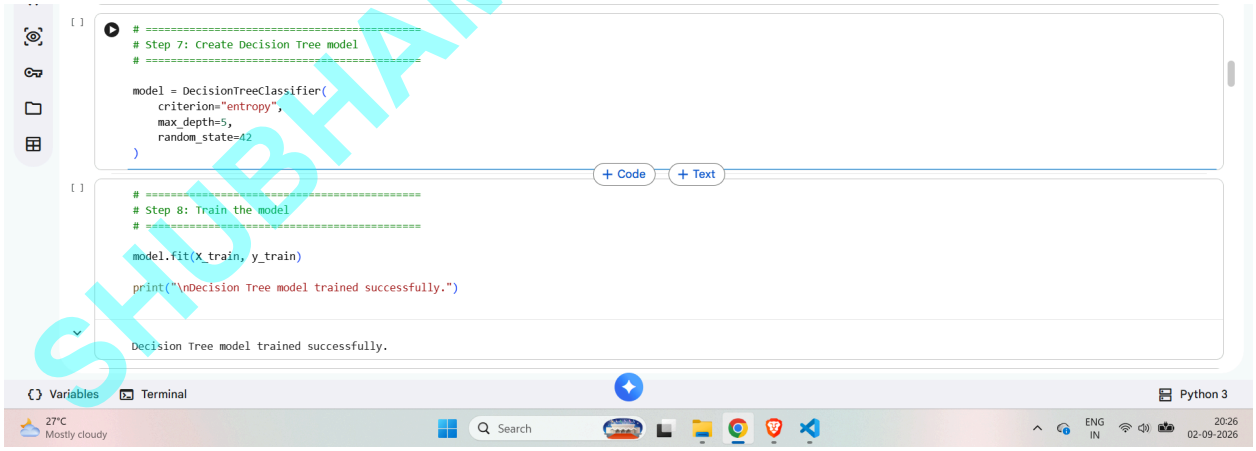


```
# ============================================
# Step 6: Split dataset into training
# and testing data
# ============================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining records:", X_train.shape[0])
print("Testing records:", X_test.shape[0])
```

```
Training records: 8000
Testing records: 2000
```

Create Decision Tree model

Train the model



```
# ============================================
# Step 7: Create Decision Tree model
# ============================================

model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)
```

```
# ============================================
# Step 8: Train the model
# ============================================

model.fit(X_train, y_train)

print("\nDecision Tree model trained successfully.")
```

```
Decision Tree model trained successfully.
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

Make predictions

Calculate accuracy

```
# ============================================
# Step 9: Make predictions
# ============================================

y_pred = model.predict(X_test)

print("\nPredictions:")
print(y_pred[:20])
```

```
Predictions:
[0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

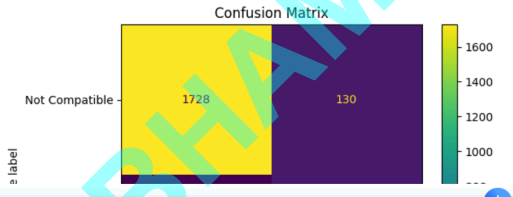
```
# ============================================
# Step 10: Calculate accuracy
# ============================================

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

```
Model Accuracy: 0.933
Accuracy Percentage: 93.30000000000001 %
```

Confusion Matrix

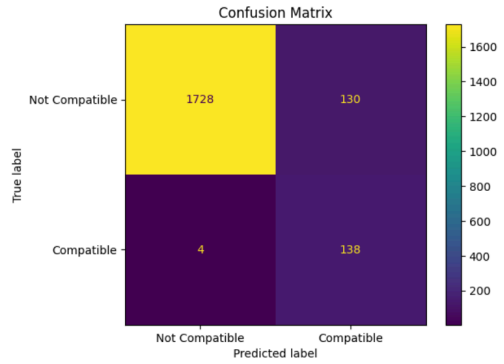
```
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Compatible", "Compatible"]
).plot()

plt.title("Confusion Matrix")
plt.show()
```

```
Confusion Matrix:
[[1728  130]
 [   4  138]]
```



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
Confusion Matrix:
[[1728  130]
 [   4  138]]
```



Classification Report

```
# ==========================================
# Step 12: Classification Report
# ==========================================

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Not Compatible", "Compatible"]
    )
)
```

```
Classification Report:
                precision    recall  f1-score   support

Not Compatible       1.00      0.93      0.96      1858
    Compatible       0.51      0.97      0.67       142

      accuracy                           0.93      2000
     macro avg       0.76      0.95      0.82      2000
  weighted avg       0.96      0.93      0.94      2000
```

Feature Importance

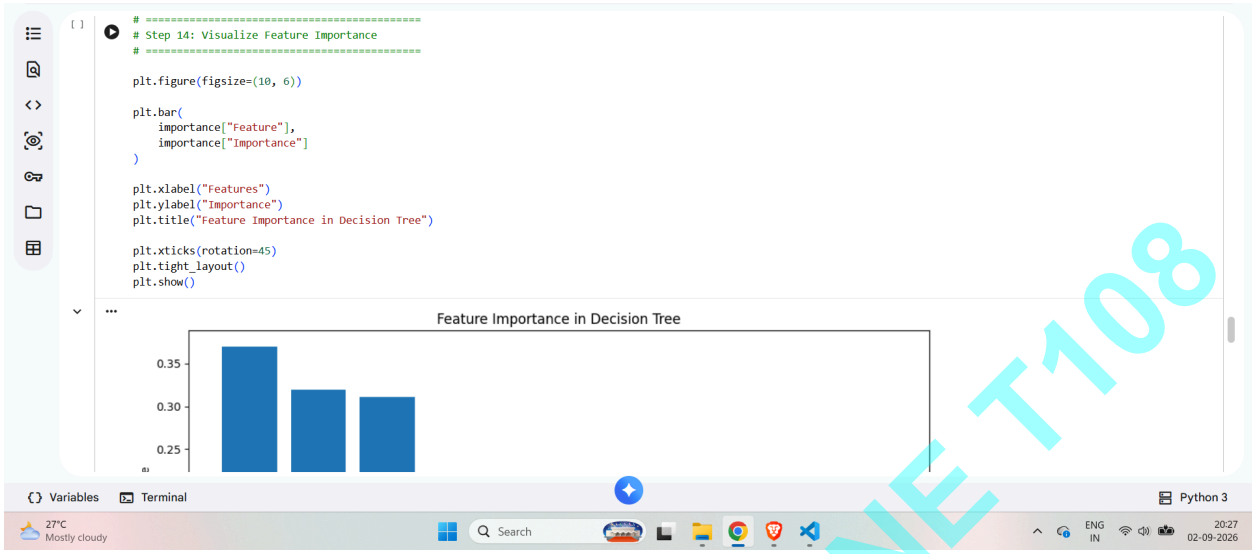
```
# ==========================================
# Step 13: Feature Importance
# ==========================================

importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance:")
print(importance)
```

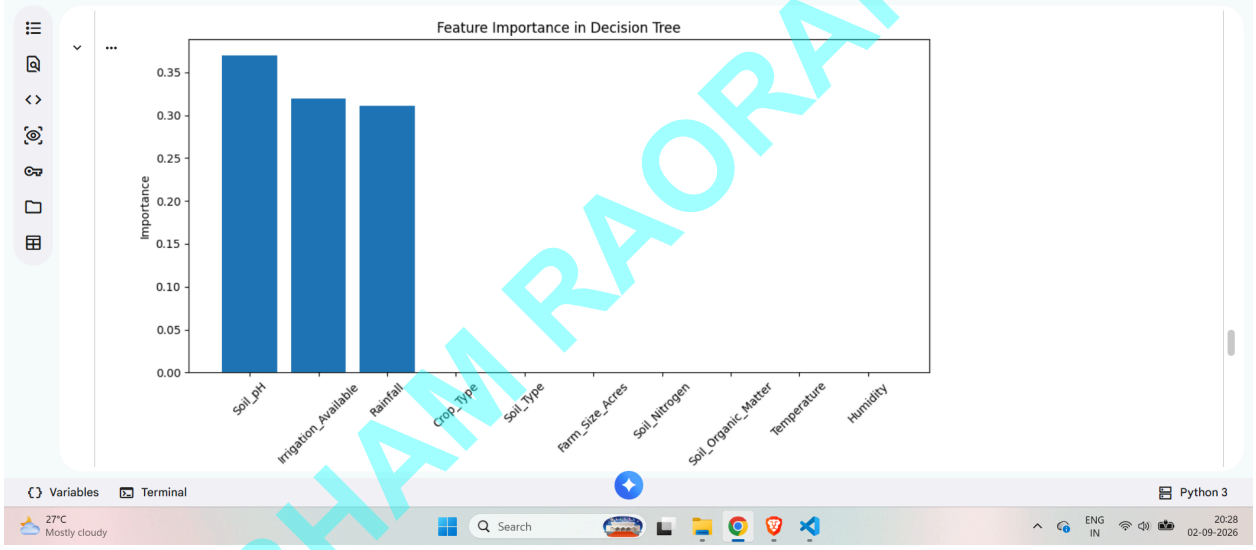
```
Feature Importance:
                Feature  Importance
4               Soil_pH    0.369776
3  Irrigation_Available    0.319038
8              Rainfall    0.311186
0             Crop_Type    0.000000
1             Soil_Type    0.000000
2       Farm_Size_Acres    0.000000
5          Soil_Nitrogen    0.000000
6   Soil_Organic_Matter    0.000000
7           Temperature    0.000000
9              Humidity    0.000000
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

Visualize Feature Importance

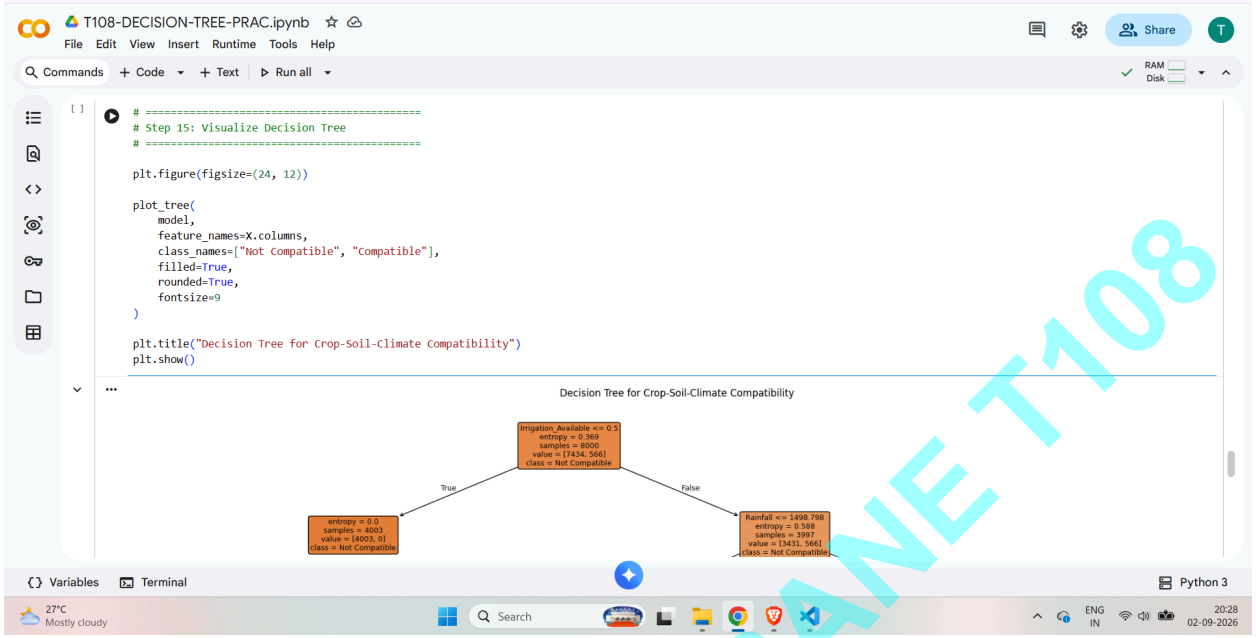
```
# ==========================================
# Step 14: Visualize Feature Importance
# ==========================================

plt.figure(figsize=(10, 6))

plt.bar(
    importance["Feature"],
    importance["Importance"]
)

plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance in Decision Tree")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

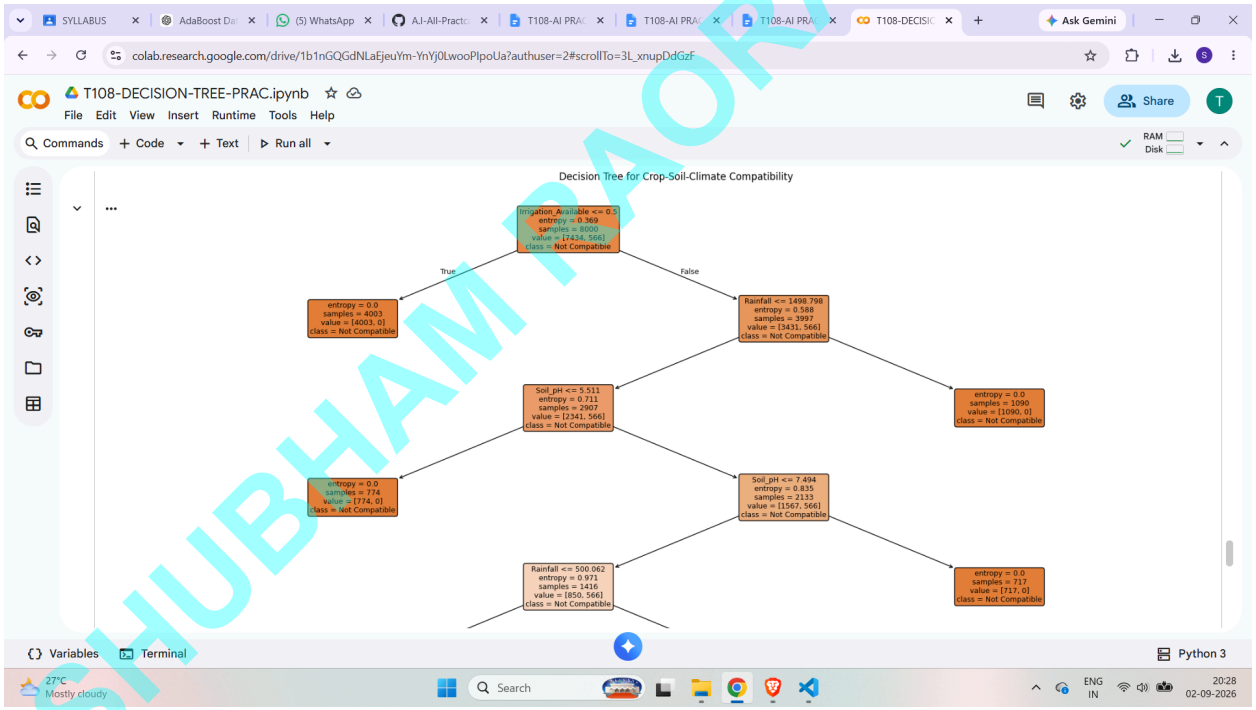




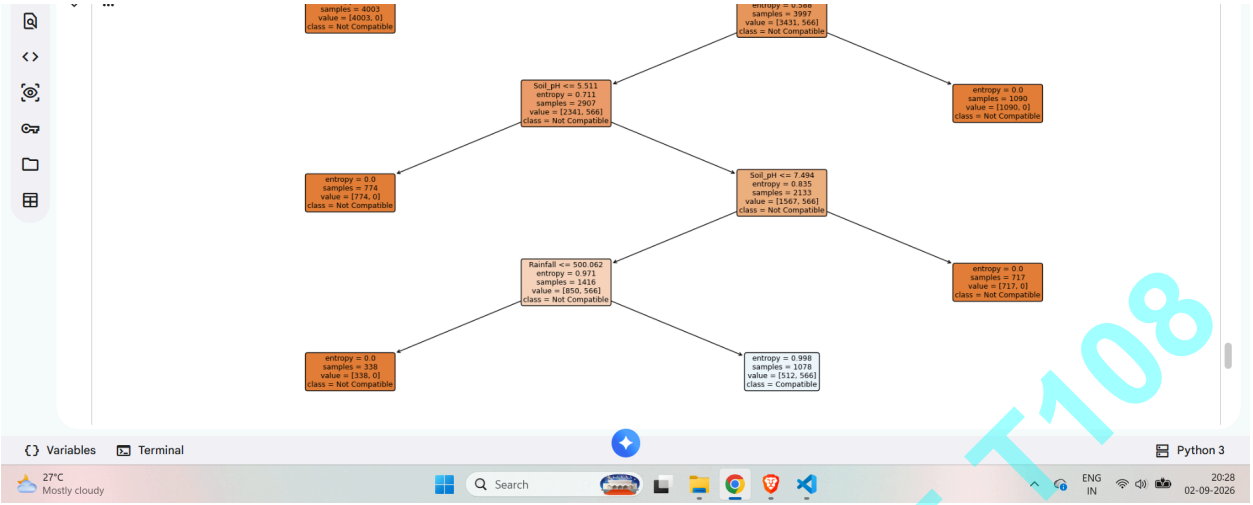
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

Visualize Decision Tree





SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI



SHUBHAM RAORANE T108